



FraudAware

Digital Wallet System

Secure Digital Payments with AI-Powered Fraud Detection

Darsh Tyagi

RA2411028030025

Rishav Kumar Gupta

RA2411028030032

Susmita Sahoo

RA2411028030041



**Fraud
Detection**

Problem Statement & Objective

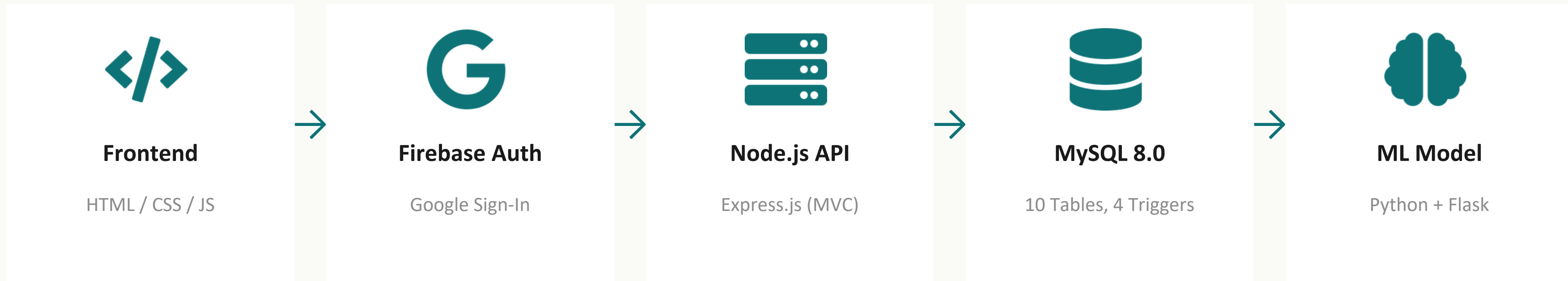
The Problem

- Digital payment fraud increased 35% year-over-year
- Rule-based systems fail against evolving attacks
- ₹1.2 lakh crore lost to UPI fraud in India (2024)
- Traditional banks take 24-48 hours to detect fraud
- No real-time behavioral analysis in most wallets

Our Solution

- ML model analyzes 15 behavioral features per transaction
- Real-time fraud scoring (0-100%) before money moves
- Detects 7 real-world attack patterns automatically
- Auto-blocks high-risk transactions via MySQL triggers
- ACID-compliant stored procedures prevent data loss

System Architecture



Request Flow

- User logs in via Firebase Google Sign-In → Backend verifies JWT token
- Backend checks if email exists in MySQL USER table → Loads wallet
- Send money → 15 features computed from MySQL → Sent to ML API
- ML returns fraud score → Stored procedure executes with COMMIT/ROLLBACK
- MySQL trigger auto-creates FRAUD_ALERT if score $\geq$ 60%

Database Design — MySQL Schema

USER User_ID 7 cols	WALLET Wallet_ID 5 cols	TRANSACTION Txn_ID 11 cols	FRAUD_ALERT Alert_ID 6 cols	DEVICE Device_ID 7 cols
KYC KYC_ID 8 cols	ADMIN Admin_ID 4 cols	REVIEW (Alert,Admin) 4 cols	AUDIT_LOG Log_ID 8 cols (NEW)	TXN_FEATURES Txn_ID 10 cols (NEW)

Key Constraints

- All tables normalized to 3NF — no partial or transitive dependencies
- 12 Foreign Keys enforce referential integrity across tables
- CHECK: Balance ≥ 0, Amount > 0 | UNIQUE: Email, User↔Wallet (1:1)
- ENUM types restrict status fields to valid values only
- B-tree indexes on FK columns and filtered fields for O(log n) lookups

Advanced DBMS Features

Triggers (4)

- Auto-create FRAUD_ALERT on high score
- Auto-log balance changes to AUDIT_LOG
- Block transactions from frozen wallets
- Fires on INSERT and UPDATE events

Views (3)

- vw_fraud_summary — 5-table JOIN
- vw_user_wallet — USER + WALLET
- vw_transaction_history — full details
- Simplify complex JOINS for API layer

Stored Procedures (2)

- sp_send_money — ACID fund transfer
- START TRANSACTION / COMMIT / ROLLBACK
- SELECT...FOR UPDATE (pessimistic lock)
- Validates balance, status, fraud score

ACID: Atomicity (all-or-nothing) · Consistency (valid state) · Isolation (no race conditions) · Durability (permanent)

ML Fraud Detection Engine

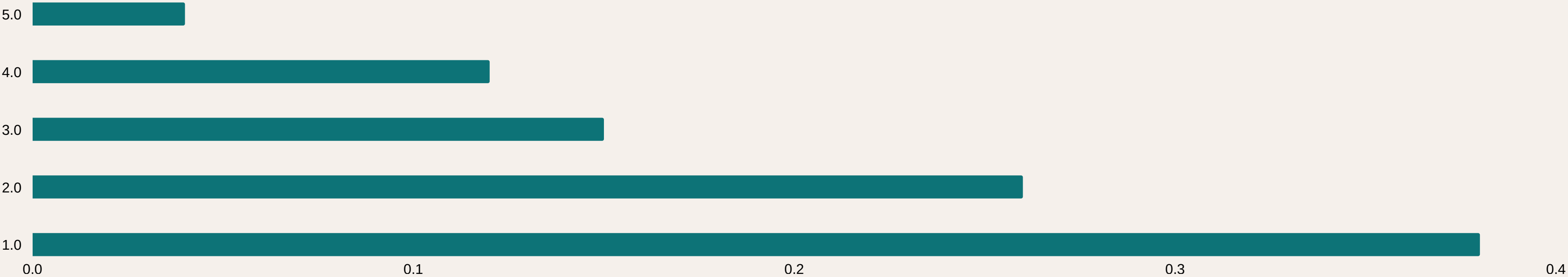
Model Specifications

- Gradient Boosted Trees (300 trees, depth 7)
- 150,000 synthetic training transactions
- 15 behavioral features per transaction
- Output: fraud score 0-100% + risk level
- Served via Flask REST API on port 5001

7 Attack Patterns Detected

- Account Takeover — credential theft, wallet drain
- Card Testing — rapid micro-transactions
- Velocity Attack — burst transfers to bypass limits
- Smurfing — structured amounts under limits
- Social Engineering, New Account, Sleeper Fraud

Top Features by Importance



15 ML Features — Computed from MySQL

#	Feature	SQL Query	Why It Matters
1	amount	Direct from request	Transaction size
2	sender_txn_count_1h	COUNT(*) WHERE time ≥ 1h ago	Burst detection
3	sender_txn_count_24h	COUNT(*) WHERE time ≥ 24h ago	Daily anomaly
4	sender_avg_amount	AVG(Amount) for sender	User baseline
5	amount_deviation	(amount - avg) / stddev	Statistical anomaly
6	is_new_receiver	COUNT sender→receiver = 0?	First-time = riskier
7	hour_of_day	NOW() hour extraction	Night = suspicious
8	is_night_transaction	hour ≥ 23 OR hour ≤ 4	Night flag
9	device_is_trusted	MAX(Is_Trusted) from DEVICE	Known device?
10	account_age_days	DATEDIFF(NOW, Created)	New = riskier
11	amount_to_avg_ratio	amount / avg_amount	How unusual?
12	txn_velocity_score	count_1h×3 + count_24h×0.5	Speed metric
13	receiver_risk_score	new_rcv × (amt + night)	Composite risk
14	amount_round_flag	amount % 1000 == 0?	Round = suspicious
15	time_since_last_txn	TIMESTAMPDIFF(MINUTE)	Rapid = suspicious

Dynamic Fraud Scoring — Demo Results

Scenario	Score	Risk	Action
₹500 → known receiver, daytime	0%	LOW	✓ Approved
₹500 → 5th txn this hour (burst)	96%	CRITICAL	✗ Blocked
₹500 → 10th txn this hour (velocity)	98%	CRITICAL	✗ Blocked
₹50,000 → known person, daytime	0%	LOW	✓ Approved
₹50,000 → new person, 3AM, new account	100%	CRITICAL	✗ Blocked
₹100 × 15 rapid txns (card testing)	100%	CRITICAL	✗ Blocked

Key: Same ₹50,000 gets 0% to a known person but 100% to a new person at 3AM — the model learns context, not just amount thresholds.

REST API & Firebase Integration

Method	Endpoint	MySQL Operations
POST	/api/send-money	5 feature queries + CALL sp_send_money
POST	/api/add-money	CALL sp_add_money (stored procedure)
GET	/api/wallet	SELECT from vw_user_wallet (view)
GET	/api/transactions	Multi-table JOIN with WHERE/ORDER/LIMIT
GET	/api/fraud-alerts	JOIN: FRAUD_ALERT ↔ TRANSACTION ↔ USER
POST	/api/fraud-check	ML prediction only (no DB write)
POST	/api/check-email	SELECT from USER table (auth gate)

Authentication

- Firebase Google Sign-In → JWT token → Backend verifies via Admin SDK
- Email must exist in MySQL USER table — simulates KYC compliance
- Demo mode available with X-Demo-User-Email header (no Firebase needed)

Conclusion & Key Takeaways

10

MySQL Tables

15

ML Features

4

DB Triggers

150K

Training Samples

- Full-stack fintech system: Firebase → Node.js → MySQL → Python ML
- ACID-compliant transactions with stored procedures and pessimistic locking
- Real-time fraud detection using Gradient Boosted Trees (150K samples)
- Database triggers automate fraud alerting — zero application code needed
- Dynamic scoring adapts to each user's behavioral patterns from MySQL

Thank You

Vanshika · Sarthak · Armish